Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 6

по курсу «Методы оптимизации»

«Освоение многопараметрических методов»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Необходимо реализовать простейший многопараметрический метод оптимизации для двух параметров.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using LinearAlgebra
using Plots
using Colors

rosenbrock(x) = sum(100 * (x[i+1] - x[i]^2)^2 + (x[i] - 1)^2 for i in 1:length(x)-1)
schwefel(x) = 418.9829 * length(x) - sum(xi -> xi * sin(sqrt(abs(xi))), x)
rastrigin(x) = 10 * length(x) + sum(xi -> xi^2 - 10 * cos(2   * xi), x)

function grad_rosenbrock(x)
    n = length(x)
    g = zeros(n)
    for i in 1:n
        if i == 1
            g[i] = -400 * x[1] * (x[2] - x[1]^2) + 2 * (x[1] - 1)
        elseif i == n
            g[i] = 200 * (x[n] - x[n-1]^2) + 2 * (x[n] - 1)
        else
            g[i] = 200 * (x[i] - x[i-1]^2) - 400 * x[i] * (x[i+1] - x[i]^2) + 2 * (x[i] - 1)
        end
    end
    return g
end

function grad_schwefel(x)
    g = similar(x)
    for i in eachindex(x)
        xi = x[i]
        s = sqrt(abs(xi)) + 1e-20
        g[i] = xi >= 0 ? -sin(s) - xi * cos(s) / (2 * s) : -sin(s) + xi * cos(s) / (2 * s)
```

```
    end
    return g
end

grad_rastrigin(x) = [2 * xi + 20   * sin(2   * xi) for xi in x]

function golden_section(f, a, b, tol=1e-8)
    phi = (sqrt(5) - 1) / 2
    c = b - phi * (b - a)
    d = a + phi * (b - a)
    while abs(c - d) > tol
        if f(c) < f(d)
            b = d
        else
            a = c
        end
        c = b - phi * (b - a)
        d = a + phi * (b - a)
    end
    return (a + b) / 2
end

function minimize_2d(f, l0_range, l1_range, max_inner=50)
    l0, l1 = (l0_range[1] + l0_range[2]) / 2, (l1_range[1] + l1_range[2]) / 2
    for _ in 1:max_inner
        l0 = golden_section(x -> f([x, l1]), l0_range[1], l0_range[2])
        l1 = golden_section(x -> f([l0, x]), l1_range[1], l1_range[2])
    end
    return [l0, l1]
end

function mille_contrell(f, grad_f, x0, eps; max_iter=10000, zero_lambda1=false)
    n = length(x0)
    x = copy(x0)
    x_prev = copy(x0)
    delta_x = zeros(n)
    traj = [copy(x)]
    iter_count = 0

    for k in 1:max_iter
        iter_count = k
```

```
        if k > 0 && k % (n + 1) == 0
            delta_x = zeros(n)
        end
        g = grad_f(x)
        if norm(g) < eps
            break
        end

        if zero_lambda1
            delta_x = zeros(n)
        end
        obj(lambda) = f(x - lambda[1] * g + lambda[2] * delta_x)
        r = max(0.1, min(1.0, norm(delta_x) / (norm(g) + 1e-10)))
        l0_range = (0.0, r * 2)
        l1_range = (-r, r)
        l_opt = zero_lambda1 ? [golden_section(t -> obj([t, 0.0]), l0_range[1], l0_range[2]), 0.0] : minimize_2d(obj, l0_range, l1_range)

        x_new = x - l_opt[1] * g + l_opt[2] * delta_x
        delta_x = x_new - x
        x_prev = x
        x = x_new
        push!(traj, copy(x))

        if norm(delta_x) < eps
            break
        end
    end

    return x, f(x), traj, iter_count
end

function draw_trajectory_2d(f, traj, title_str, filename)
    xs = [p[1] for p in traj]
    ys = [p[2] for p in traj]
    fv = [f(p) for p in traj]
    rng = max(maximum(fv) - minimum(fv), 1e-10)
    span_x = maximum(xs) - minimum(xs)
    span_y = maximum(ys) - minimum(ys)
    pad = 0.2 * max(span_x, span_y, 0.5)
    xlim = (minimum(xs) - pad, maximum(xs) + pad)
    ylim = (minimum(ys) - pad, maximum(ys) + pad)
    xg = range(xlim[1], xlim[2], length=80)
    yg = range(ylim[1], ylim[2], length=80)
```

```
    plt = contour(xg, yg, (x, y) -> f([x, y]), levels=25, color=cgrad(:viridis), colorbar=true, legend=false, xlabel="x_1", ylabel="x_2", title=title_str, xlims=xlim, ylims=ylim)
    for i in 1:length(traj)-1
        t = (fv[i] - minimum(fv)) / rng
        col = RGB(0.2 + 0.6 * (1 - t), 0.3 + 0.5 * (1 - t), 0.8 - 0.3 * t)
        plot!(plt, xs[i:i+1], ys[i:i+1], color=col, lw=3, label="")
    end
    scatter!(plt, [xs[1]], [ys[1]], color=:green, ms=8, marker=:circle, label="")
    scatter!(plt, [xs[end]], [ys[end]], color=:red, ms=8, marker=:circle, label="")
    savefig(plt, filename)
    display(plt)
end

eps = 1e-6

funcs = [
    ("                    ", rosenbrock, grad_rosenbrock, [0.0, 0.0]),
    ("                ", schwefel, grad_schwefel, [350.0, 400.0]),
    ("                    ", rastrigin, grad_rastrigin, [3.0, 3.0])
]

results = []
println("                        ")
for (name, f, grad_f, x0) in funcs
    x, fv, traj, it = mille_contrell(f, grad_f, x0, eps)
    println(name, ":")
    println("  x* = ", round.(x, digits=6))
    println("  f(x*) = ", round(fv, digits=10))
    println("                        = ", it)
    push!(results, (name=name, f=f, traj=traj))
end

println("\n lambda_1 = 0")
for (name, f, grad_f, x0) in funcs
    x, fv, traj, it = mille_contrell(f, grad_f, x0, eps; zero_lambda1=true)
    println(name, ":")
    println("  x* = ", round.(x, digits=6))
    println("  f(x*) = ", round(fv, digits=10))
    println("                        = ", it)
```

```
end

draw_trajectory_2d(results[1].f, results[1].traj, "
                        ", "lab6_rosenbrock.png")
draw_trajectory_2d(results[2].f, results[2].traj, "                    ",
    "lab6_schwefel.png")
draw_trajectory_2d(results[3].f, results[3].traj, "
                        ", "lab6_rastrigin.png")
```

Результат представлен на рисунках 1– 3.

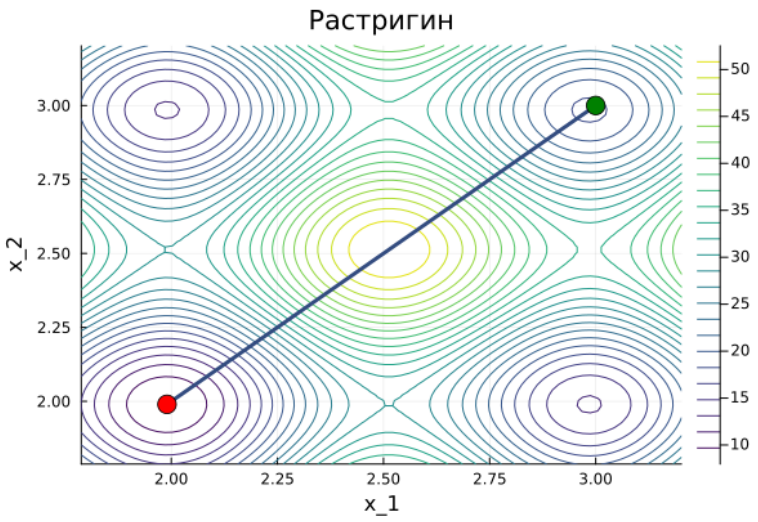

Рис. 1 — Результат

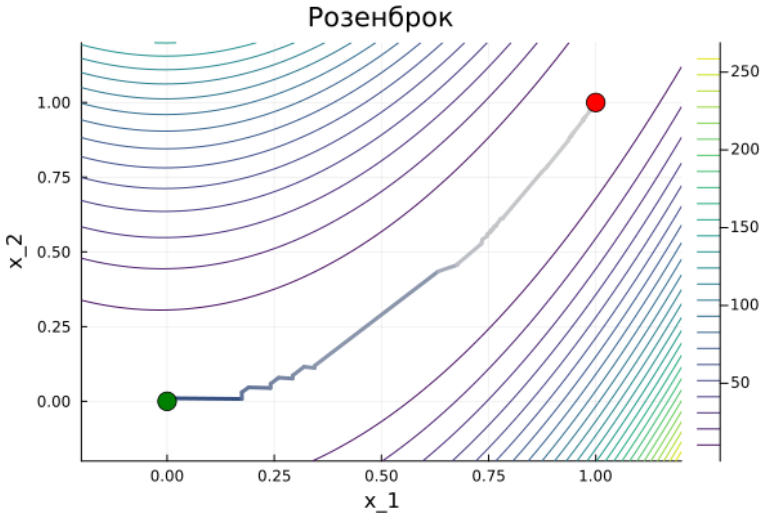

Рис. 2 — Результат

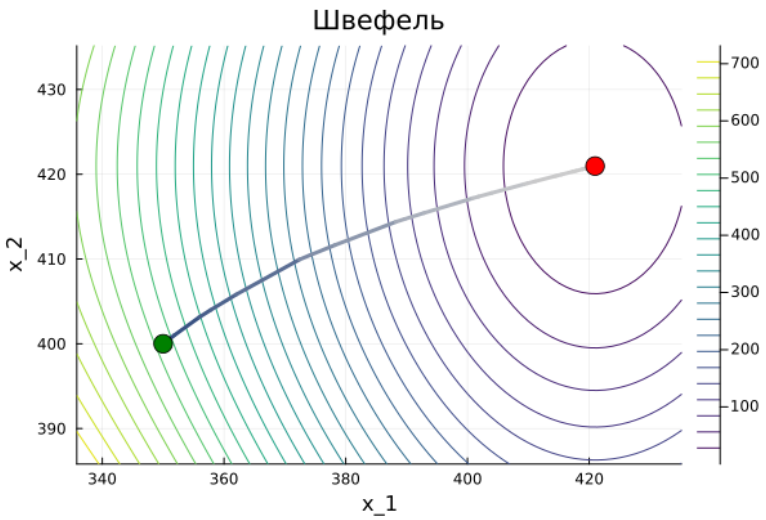

Рис. 3 — Результат